

Exhaustive Search Method

% Bracket the minimum of the function $\phi(\phi) = \phi^2 + 54/\phi$ using an exhaustive search method in the interval (0,5) by considering only 10 intermediate points

% Define the function

```
f = @(x) x.^2 + 54./ x;
```

% Define the interval and discretization

```
a = 0; % Start just above 0 to avoid division by zero
```

```
b = 5;
```

```
n = 10;
```

```
dx = (b - a) / n;
```

% Initialize variables

```
x1 = a;
```

```
x2 = x1 + dx;
```

```
x3 = x2 + dx;
```

% Initialize best value

```
fmin = Inf;
```

```
lower = 0;
```

```
upper = 0;
```

% Perform the grid search

```
while x3 <= b
```

```
    f1 = f(x1);
```

```
    f2 = f(x2);
```

```
    f3 = f(x3);
```

```
    % Check if x2 is a local minimum
```

```
    if f1 >= f2 && f2 <= f3
```

```
        if f2 < fmin
```

```
            fmin = f2;
```

```
            lower = x1;
```

```
            upper = x3;
```

```
        end
```

```
    end
```

```
    % Update x1, x2, and x3
```

```
    x1 = x2;
```

```
    x2 = x3;
```

```
    x3 = x2 + dx;
```

```
end
```

```
fprintf('Minimum lies in (%.3f, %.3f)\n', lower, upper);
```

Minimum lies in (2.500, 3.500)

```
x=(lower+upper)/2;
```

```
fprintf('Optimized values:\n x=%.3f \n minimum f=%.3f', x,f(x))
```

Optimized values:

x=3.000

minimum f=27.000

Alternative code for the Exhaustive Search Method

```
% Function definition
f = @(x) x.^2 + 54./x;
% Interval and number of points
a = 0; % Start of interval
b = 5; % End of interval
n = 10; % Number of intermediate points
% Step size
delta_x = (b - a) / (n);
% Initialize variables
x_lower = a;
f_lower = f(a);
min_found = false;
% Exhaustive Search Loop
for i = 1: n
    x_current = a + i * delta_x;
    f_current = f(x_current);
    % Compare current function value with previous one
    if f_current < f_lower
        x_lower = x_current;
        f_lower = f_current; min_found = true;
    else
        % Stop if we find the function value starts increasing
        if min_found
            x_upper = x_current;
            break;
        end
    end
end
% Compute function values at the interval bounds
f_lower_bound = f(x_lower - delta_x);
f_upper_bound = f(x_upper);
% Display the final result
fprintf('\nThe minimum lies in the range [%.3f, %.3f]\n', x_lower - delta_x,
x_upper);
```

The minimum lies in the range [2.500, 3.500]

```
fprintf('Function value at lower bound: %.3f\n', f_lower_bound);
```

Function value at lower bound: 27.850

```
fprintf('Function value at upper bound: %.3f\n', f_upper_bound);
```

Function value at upper bound: 27.679

```
fprintf('Function value at the minimum: %.3f\n', f_lower);
```

Function value at the minimum: 27.000

Qn1

```

% Exhaustive Search Method for  $f(x) = x^3 - 4x^2 + 6x - \exp(0.1x)$ 
f = @(x) x.^3 - 4*x.^2 + 6*x - exp(0.1*x);

a = 1; % Start of interval
b = 4; % End of interval
n = 20; % Number of intermediate points
% Step size
delta_x = (b - a) / (n);
% Initialize variables
x_lower = a;
f_lower = f(a);
min_found = false;
% Exhaustive Search Loop
for i = 1: n
    x_current = a + i * delta_x;
    f_current = f(x_current);
    % Compare current function value with previous one
    if f_current < f_lower
        x_lower = x_current;
        f_lower = f_current; min_found = true;
    else
        % Stop if we find the function value starts increasing
        if min_found
            x_upper = x_current;
            break;
        end
    end
end
% Compute function values at the interval bounds
f_lower_bound = f(x_lower - delta_x);
f_upper_bound = f(x_upper);
% Display the final result
fprintf('\nThe minimum lies in the range [%.3f, %.3f]\n', x_lower - delta_x,
x_upper);

```

The minimum lies in the range [0.850, 3.500]

```
fprintf('Function value at lower bound: %.3f\n', f_lower_bound);
```

Function value at lower bound: 1.735

```
fprintf('Function value at upper bound: %.3f\n', f_upper_bound);
```

Function value at upper bound: 13.456

```
fprintf('Function value at the minimum: %.3f\n', f_lower);
```

Function value at the minimum: 1.895

Qn2

```

% Exhaustive Search Method for  $f(x) = \sqrt{x^2 + 1} - 2x$ 
f = @(x) sqrt(x.^2 + 1) - 2*x;

```

```

a = -2.5;
b = 1;
n = 15;
% Step size
delta_x = (b - a) / (n);
% Initialize variables
x_lower = a;
f_lower = f(a);
min_found = false;
% Exhaustive Search Loop
for i = 1: n
    x_current = a + i * delta_x;
    f_current = f(x_current);
    % Compare current function value with previous one
    if f_current < f_lower
        x_lower = x_current;
        f_lower = f_current; min_found = true;
    else
        % Stop if we find the function value starts increasing
        if min_found
            x_upper = x_current;
            break;
        end
    end
end
% Compute function values at the interval bounds
f_lower_bound = f(x_lower - delta_x);
f_upper_bound = f(x_upper);
% Display the final result
fprintf('\nThe minimum lies in the range [%.3f, %.3f]\n', x_lower - delta_x,
x_upper);

```

The minimum lies in the range [0.767, 3.500]

```
fprintf('Function value at lower bound: %.3f\n', f_lower_bound);
```

Function value at lower bound: 0.233

```
fprintf('Function value at upper bound: %.3f\n', f_upper_bound);
```

Function value at upper bound: 2.500

```
fprintf('Function value at the minimum: %.3f\n', f_lower);
```

Function value at the minimum: 0.000

Qn3

```

% Exhaustive Search Method for f(x) = x^3 - 4x^2 + 6x - exp(0.1x)
f = @(x) (x.^2)*log(x+1) - (1/(x+1));

a = 0.5; % Start of interval
b = 2.5; % End of interval

```

```

n = 10; % Number of intermediate points
% Step size
% Step size
delta_x = (b - a) / (n);
% Initialize variables
x_lower = a;
f_lower = f(a);
min_found = false;
% Exhaustive Search Loop
for i = 1: n
    x_current = a + i * delta_x;
    f_current = f(x_current);
    % Compare current function value with previous one
    if f_current < f_lower
        x_lower = x_current;
        f_lower = f_current; min_found = true;
    else
        % Stop if we find the function value starts increasing
        if min_found
            x_upper = x_current;
            break;
        end
    end
end
% Compute function values at the interval bounds
f_lower_bound = f(x_lower - delta_x);
f_upper_bound = f(x_upper);
% Display the final result
fprintf('\nThe minimum lies in the range [%.3f, %.3f]\n', x_lower - delta_x,
x_upper);

```

The minimum lies in the range [0.300, 3.500]

```
fprintf('Function value at lower bound: %.3f\n', f_lower_bound);
```

Function value at lower bound: -0.746

```
fprintf('Function value at upper bound: %.3f\n', f_upper_bound);
```

Function value at upper bound: 18.203

```
fprintf('Function value at the minimum: %.3f\n', f_lower);
```

Function value at the minimum: -0.565

Qn4

```

% Bounding Phase Method for  $f(x) = (1-x)^4 - (2x+10)^2$ 
f = @(x) (1-x).^4 - (2*x+10).^2;

x0 = 0.5;
delta = 0.6;

% Evaluate at initial points
f0 = f(x0);

```

```

f1 = f(x0 + delta);
f_1 = f(x0 - delta);

% Determine search direction
if f_1 >= f0 && f1 >= f0
    %fprintf('Minimum lies between [%.4f, %.4f]\n', x0-delta, x0+delta);
    return;
elseif f_1 <= f0 && f0 <= f1
    delta = -delta;
elseif f_1 >= f0 && f0 >= f1
    % delta is positive, do nothing
else
    error('Function is not unimodal in the neighborhood of x0.');
```

```

end

% Start bounding phase
k = 0;
x_prev = x0;
x_curr = x0 + delta;
while true
    x_next = x_curr + 2^k * delta;
    if f(x_next) > f(x_curr)
        break;
    end
    x_prev = x_curr;
    x_curr = x_next;
    k = k + 1;
end

% Bracket found
a = min(x_prev, x_next);
b = max(x_prev, x_next);
fprintf('Minimum lies between [%.4f, %.4f]\n', a, b);

```

Minimum lies between [1.7000, 5.3000]

Qn5

```

% Bounding Phase Method for  $f(x) = \exp((x-0.2)^2/2)$ 
f = @(x) exp(((x-0.2).^2)/2);

x0 = -10;
delta = 0.8;

% Evaluate at initial points
f0 = f(x0);
f1 = f(x0 + delta);
f_1 = f(x0 - delta);

% Determine search direction
if f_1 >= f0 && f1 >= f0
    %fprintf('Minimum lies between [%.4f, %.4f]\n', x0-delta, x0+delta);
    return;

```

```

elseif f_1 <= f0 && f0 <= f1
    delta = -delta;
elseif f_1 >= f0 && f0 >= f1
    % delta is positive, do nothing
else
    error('Function is not unimodal in the neighborhood of x0.');
```

end

```

% Start bounding phase
k = 0;
x_prev = x0;
x_curr = x0 + delta;
while true
    x_next = x_curr + 2^k * delta;
    if f(x_next) > f(x_curr)
        break;
    end
    x_prev = x_curr;
    x_curr = x_next;
    k = k + 1;
end

% Bracket found
a = min(x_prev, x_next);
b = max(x_prev, x_next);
fprintf('Minimum lies between [%.4f, %.4f]\n', a, b);
```

Minimum lies between [-3.6000, 15.6000]

Qn6

```

% Bounding Phase Method for  $f(x) = (x+2)^2 / (\sin(x+1)+2)$ 
f = @(x) ((x+2).^2) ./ (sin(x+1)+2);

x0 = -1.5;
delta = 0.05;

% Evaluate at initial points
f0 = f(x0);
f1 = f(x0 + delta);
f_1 = f(x0 - delta);

% Determine search direction
if f_1 >= f0 && f1 >= f0
    fprintf('Minimum lies between [%.4f, %.4f]\n', x0-delta, x0+delta);
    return;
elseif f_1 <= f0 && f0 <= f1
    delta = -delta;
elseif f_1 >= f0 && f0 >= f1
    % delta is positive, do nothing
else
    error('Function is not unimodal in the neighborhood of x0.');
```

end

```

% Start bounding phase
k = 0;
x_prev = x0;
x_curr = x0 + delta;
while true
    x_next = x_curr + 2^k * delta;
    if f(x_next) > f(x_curr)
        break;
    end
    x_prev = x_curr;
    x_curr = x_next;
    k = k + 1;
end

% Bracket found
a = min(x_prev, x_next);
b = max(x_prev, x_next);
fprintf('Minimum lies between [%.4f, %.4f]\n', a, b);

```

Minimum lies between [-2.3000, -1.7000]

Qn7.a

```

% Exhaustive Search for f(x) = sqrt(x) - sin(x^2) in (-1,2) with 15 points
f = @(x) sqrt(x) - sin(x.^2);

a = -1;
b = 2;
n = 15;
x = linspace(a, b, n+2); % 15 intermediate points means 17 total points
fx = f(x);

% Find interval containing minimum
[min_val, min_idx] = min(fx);
if min_idx == 1
    fprintf('Minimum at left endpoint: x = %.4f\n', x(1));
elseif min_idx == length(x)
    fprintf('Minimum at right endpoint: x = %.4f\n', x(end));
else
    fprintf('Minimum lies between [%.4f, %.4f]\n', x(min_idx-1), x(min_idx+1));
end

```

Minimum lies between [1.0625, 1.4375]

Qn7.b

```

% Bounding Phase Method for f(x) = sqrt(x) - sin(x^2)
f = @(x) sqrt(x) - sin(x.^2);

x0 = 1;
delta = 0.4;

```

```

% Evaluate at initial points
f0 = f(x0);
f1 = f(x0 + delta);
f_1 = f(x0 - delta);

% Determine search direction
if f_1 >= f0 && f1 >= f0
    fprintf('Minimum lies between [%.4f, %.4f]\n', x0-delta, x0+delta);
    return;
elseif f_1 <= f0 && f0 <= f1
    delta = -delta;
elseif f_1 >= f0 && f0 >= f1
    % delta is positive, do nothing
else
    error('Function is not unimodal in the neighborhood of x0.');
```

```

end

% Start bounding phase
k = 0;
x_prev = x0;
x_curr = x0 + delta;
while true
    x_next = x_curr + 2^k * delta;
    if f(x_next) > f(x_curr)
        break;
    end
    x_prev = x_curr;
    x_curr = x_next;
    k = k + 1;
end

% Bracket found
a = min(x_prev, x_next);
b = max(x_prev, x_next);
fprintf('Minimum lies between [%.4f, %.4f]\n', a, b);

```

Qn8.a

```

% Exhaustive Search for  $f(x) = (\cos(x)^2 + x)/(x - 2)$  over  $(-3, 3)$ 
f = @(x) (cos(x).^2 + x) ./ (x - 2);

a = -3;
b = 3;
n = 14; % Number of intervals
h = (b - a) / n;

x = a:h:b;
fx = f(x);

min_idx = 1;
for i = 2:length(x)-1
    if fx(i) < fx(i-1) && fx(i) < fx(i+1)
        min_idx = i;
    end
end

```

```
        break;
    end
end

fprintf('Minimum lies between [%.4f, %.4f]\n', x(min_idx-1), x(min_idx+1));

Minimum lies between [1.2857, 2.1429]
```